TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

TÀI LIỆU THỰC HÀNH

Học máy ứng dụng

Bài 7: Đánh giá mô hình

Biên soạn ThS. Nguyễn Thái Anh

Năm học 2026 – 2027

Mục lục

1 Mục tiêu bài thực hành **2**
1.1 Chuẩn bị . . . 2

2 Khi accuracy đánh lừa **3**
2.1 Đọc kết quả . . . 4

3 Ma trận nhầm lẫn **5**
3.1 Đọc kết quả . . . 6

4 Precision, recall và F1 **7**
4.1 Đọc kết quả . . . 8

5 Đổi ngưỡng quyết định **9**
5.1 Đọc bảng thứ nhất . . . 10
5.2 Chọn ngưỡng đúng cách . . . 11

6 Đường ROC và AUC **11**
6.1 Đọc kết quả . . . 13
6.2 So các mô hình đã học . . . 13

7 Kiểm định chéo nhiều thước đo và cân bằng lớp **14**
7.1 Vì sao phải chia có phân tầng . . . 15
7.2 Kiểm định chéo với thước đo khác accuracy . . . 15
7.3 Cân bằng lớp khi học . . . 15

8 Bài tập **15**

9 Phụ lục A, bảng tra nhanh **17**

10 Phụ lục B, lỗi thường gặp **18**

1. Mục tiêu bài thực hành

Sáu buổi trước, mọi mô hình đều được chấm bằng accuracy. Buổi này cho thấy khi nào accuracy đánh lừa ta, và học những thước đo nhìn riêng từng loại sai.

Sau khi hoàn thành bài này, sinh viên có khả năng:

1. So sánh mô hình với mô hình lười luôn đoán lớp đông nhất.
2. Tự đếm bốn ô TP, FP, FN, TN và đọc ma trận nhầm lẫn của scikit-learn.
3. Tính precision, recall, F1 bằng tay và bằng thư viện, đọc được `classification_report`.
4. Đổi ngưỡng quyết định, và chọn ngưỡng đúng cách bằng kiểm định chéo trên tập học.
5. Tính AUC và dùng nó để so các mô hình đã học trong môn.
6. Dùng kiểm định chéo với thước đo khác accuracy và thử cân bằng lớp khi học.

Bộ dữ liệu

Tệp `data/sinh_vien_bo_hoc.csv` gồm 1200 sinh viên, là số liệu mô phỏng dùng cho mục đích giảng dạy. Nhà trường muốn phát hiện sớm những bạn có nguy cơ bỏ học để kịp tư vấn. Chỉ khoảng 10% sinh viên bỏ học, đây chính là điều làm bài toán khó đánh giá.

Bảng 1: *Các cột của tệp `sinh_vien_bo_hoc.csv`.*

Cột	Ý nghĩa
`diem_tb`	Điểm trung bình học kỳ trước, thang 10.
`so_buoi_vang`	Số buổi vắng trong học kỳ.
`tin_chi_no`	Số tín chỉ còn nợ.
`gio_lam_them`	Số giờ làm thêm mỗi tuần.
`bo_hoc`	Nhãn cần đoán. Bằng 1 nếu bỏ học, bằng 0 nếu học tiếp.

1.1. Chuẩn bị

Vẫn môi trường của bài 1, không cài thêm gì.

```
cd C:\MayHoc
hocmay\Scripts\activate
cd bai07_danh_gia
```

Bảng 2: *Sáu tệp mã của buổi thực hành, chạy theo đúng thứ tự này.*

Mục	Tệp	Nội dung
2	`h1_do_bua.py`	Accuracy đánh lừa trên dữ liệu lệch lớp
3	`h2_ma_tran.py`	Tự đếm bốn ô của ma trận nhầm lẫn
4	`h3_precision_recall.py`	Precision, recall, F1
5	`h4_nguong.py`	Đổi ngưỡng và chọn ngưỡng đúng cách
6	`h5_roc_auc.py`	Đường ROC, AUC và so các mô hình
7	`h6_can_bang.py`	Kiểm định chéo nhiều thước đo và cân bằng lớp

2. Khi accuracy đánh lừa

Bước đầu tiên của mọi bài toán phân loại là đếm số mẫu mỗi lớp. Sau đó so mô hình với **mô hình lười**, tức mô hình ai cũng đoán là lớp đông nhất. scikit-learn có sẵn mô hình này là `DummyClassifier`.

```python
# -*- coding: utf-8 -*-
"""Bước 1: thấy accuracy đánh lừa trên dữ liệu lệch lớp.

Chú thích viết tiếng Việt có dấu, phần in ra màn hình viết không dấu.
"""

import pandas as pd
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/sinh_vien_bo_hoc.csv")
print("Kich thuoc bang:", df.shape)
print(df.head())
print()
print("So sinh vien moi lop:")
print(df["bo_hoc"].value_counts().to_string())
print(f"Ty le bo hoc: {df['bo_hoc'].mean():.3f}")
print()

X = df.drop(columns="bo_hoc")
y = df["bo_hoc"]
# stratify=y giữ đúng tỷ lệ bỏ học ở cả tập học lẫn tập kiểm tra
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)
print(f"Tap kiem tra: {len(y_te)} ban, {y_te.sum()} ban bo hoc")
print()

# Mô hình lười: ai cũng đoán là lớp đông nhất
luoi = DummyClassifier(strategy="most_frequent").fit(X_tr, y_tr)
du_luoi = luoi.predict(X_te)

lr = make_pipeline(StandardScaler(), LogisticRegression())
lr.fit(X_tr, y_tr)
du_lr = lr.predict(X_te)

for ten, du in [("Mo hinh luoi", du_luoi), ("Hoi quy logistic", du_lr)]:
    acc = accuracy_score(y_te, du)
    tim_ra = ((du == 1) & (y_te == 1)).sum()
    print(f"{ten:17s} accuracy {acc:.4f}, "
```

```
          f"tim ra {tim_ra} tren {y_te.sum()} ban bo hoc")
```

```
python code/h1_do_bua.py
```

```
Kich thuoc bang: (1200, 5)
   diem_tb  so_buoi_vang  tin_chi_no  gio_lam_them  bo_hoc
0      8.0             3           0             0       0
1      8.9             5           1             7       0
2      3.3             3           1             4       0
3      6.4             3           0             7       0
4      7.9             3           5            23       0

So sinh vien moi lop:
bo_hoc
0    1081
1     119
Ty le bo hoc: 0.099

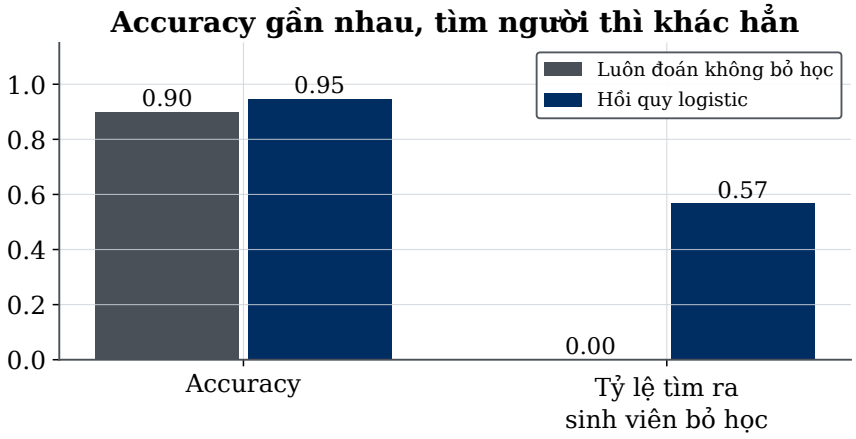
Tap kiem tra: 300 ban, 30 ban bo hoc

Mo hinh luoi      accuracy 0.9000, tim ra 0 tren 30 ban bo hoc
Hoi quy logistic  accuracy 0.9467, tim ra 17 tren 30 ban bo hoc
```

2.1. Đọc kết quả

Chỉ 119 trên 1200 sinh viên bỏ học, tỷ lệ 0.099. Nhờ `stratify=y`, tập kiểm tra có đúng 30 trên 300 bạn bỏ học, giữ nguyên tỷ lệ đó.

Mô hình lười đạt accuracy 0.9000 mà không tìm ra một bạn bỏ học nào. Hồi quy logistic đạt 0.9467, chỉ hơn mô hình lười chưa tới 5 điểm phần trăm, nhưng tìm ra được 17 trên 30 bạn.



Hình 1: *Accuracy và tỷ lệ tìm ra sinh viên bỏ học của hai mô hình trên cùng tập kiểm tra.*

Mức sàn của accuracy

Với dữ liệu lệch lớp, accuracy của mô hình lười bằng đúng tỷ lệ của lớp đông nhất. Mọi mô hình phải được so với mức sàn này, không phải so với con số 0. Accuracy 0.9 nghe rất cao,

nhưng ở bài này nó chỉ là mức sàn.

3. Ma trận nhầm lẫn

Mỗi dự đoán của mô hình rơi vào đúng một trong bốn ô, tuỳ nhãn thật và nhãn đoán.

Bảng 3: *Bốn ô của ma trận nhầm lẫn, theo đúng cách scikit-learn sắp xếp.*

	Đoán không bỏ học (0)	**Đoán bỏ học (1)**
Thật không bỏ (0)	TN, đoán đúng lớp 0	FP, báo động nhầm
Thật bỏ học (1)	FN, bỏ sót	TP, đoán đúng lớp 1

Tệp mã dưới đây tự đếm bốn ô bằng numpy, rồi so với hàm `confusion_matrix`.

```python
# -*- coding: utf-8 -*-
"""Bước 2: tự đếm bốn ô của ma trận nhầm lẫn, rồi so với thư viện."""

import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/sinh_vien_bo_hoc.csv")
X = df.drop(columns="bo_hoc")
y = df["bo_hoc"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

lr = make_pipeline(StandardScaler(), LogisticRegression())
du = lr.fit(X_tr, y_tr).predict(X_te)
that = y_te.to_numpy()

# Tự đếm: mỗi ô là một cặp (nhãn thật, nhãn đoán)
tp = np.sum((that == 1) & (du == 1))
fp = np.sum((that == 0) & (du == 1))
fn = np.sum((that == 1) & (du == 0))
tn = np.sum((that == 0) & (du == 0))
print("Tu dem:")
print(f"  TP = {tp}  (doan bo hoc, dung la bo hoc)")
print(f"  FP = {fp}  (bao dong nham)")
print(f"  FN = {fn}  (bo sot)")
print(f"  TN = {tn}  (doan khong bo, dung la khong bo)")
print(f"  Tong = {tp + fp + fn + tn}")
print()

```

```
m = confusion_matrix(that, du)
print("confusion_matrix cua scikit-learn:")
print(m)
print("Thu tu khi trai ra: TN, FP, FN, TP =", m.ravel())
print()

acc = (tp + tn) / (tp + fp + fn + tn)
print(f"Accuracy tinh tu bon o: ({tp} + {tn}) / {len(that)} = {acc:.4f}")
print(f"So ban bi bao co nguy co: TP + FP = {tp + fp}")
```

```
python code/h2_ma_tran.py
```

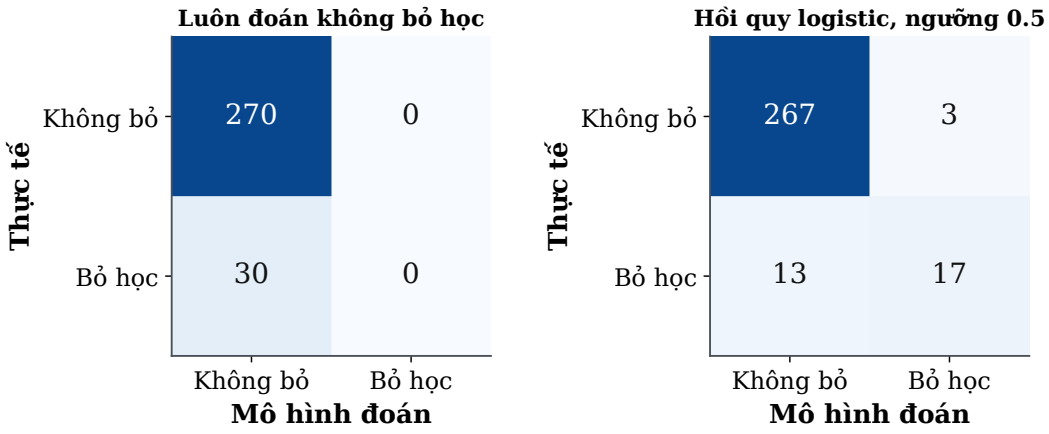
```
Tu dem:
  TP = 17  (doan bo hoc, dung la bo hoc)
  FP = 3  (bao dong nham)
  FN = 13  (bo sot)
  TN = 267  (doan khong bo, dung la khong bo)
  Tong = 300

confusion_matrix cua scikit-learn:
[[267   3]
 [ 13  17]]
Thu tu khi trai ra: TN, FP, FN, TP = [267   3  13  17]

Accuracy tinh tu bon o: (17 + 267) / 300 = 0.9467
So ban bi bao co nguy co: TP + FP = 20
```

3.1. Đọc kết quả

Bốn ô tự đếm khớp đúng với ma trận của scikit-learn: TN = 267, FP = 3, FN = 13, TP = 17, tổng 300. Accuracy tính từ bốn ô là $(17 + 267)/300 \approx 0.9467$, đúng như mục 2.



Hình 2: *Ma trận nhầm lẫn của mô hình lười và của hồi quy logistic trên cùng tập kiểm tra.*

Mô hình logistic báo 20 bạn có nguy cơ, trong đó 17 bạn bỏ học thật. Nhưng nó bỏ sót 13 bạn, gần một nửa số bạn bỏ học.

Thứ tự khi trải ma trận ra

`confusion_matrix(...).ravel()` trả về theo thứ tự TN, FP, FN, TP. Viết `tn, fp, fn, tp = m.ravel()` là đúng. Viết `tp, fp, fn, tn = m.ravel()` là một lỗi rất hay gặp, và mọi con số sau đó đều sai.

4. Precision, recall và F1

Ba thước đo cho lớp dương

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN} \qquad F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precision trả lời câu hỏi *báo thì có đúng không*. Recall trả lời câu hỏi *có sót ai không*. F1 là trung bình điều hoà của hai con số, chỉ cao khi cả hai cùng cao.

```python
# -*- coding: utf-8 -*-
"""Bước 3: precision, recall, F1 tính tay và tính bằng thư viện."""

import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (classification_report, confusion_matrix,
                             f1_score, precision_score, recall_score)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/sinh_vien_bo_hoc.csv")
X = df.drop(columns="bo_hoc")
y = df["bo_hoc"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

lr = make_pipeline(StandardScaler(), LogisticRegression())
du = lr.fit(X_tr, y_tr).predict(X_te)
tn, fp, fn, tp = confusion_matrix(y_te, du).ravel()

p = tp / (tp + fp)
r = tp / (tp + fn)
f1 = 2 * p * r / (p + r)
print("Tinh tay tu bon o:")
print(f"  Precision = {tp} / ({tp} + {fp}) = {p:.4f}")
print(f"  Recall    = {tp} / ({tp} + {fn}) = {r:.4f}")
print(f"  F1        = 2 x {p:.4f} x {r:.4f} / ({p:.4f} + {r:.4f})"
      f" = {f1:.4f}")
print()

```

```
print("Tinh bang scikit-learn:")
print(f"  Precision = {precision_score(y_te, du):.4f}")
print(f"  Recall    = {recall_score(y_te, du):.4f}")
print(f"  F1        = {f1_score(y_te, du):.4f}")
print()

print("Bao cao day du:")
print(classification_report(y_te, du, digits=4,
                            target_names=["khong bo", "bo hoc"]))
```

```
python code/h3_precision_recall.py
```

```
Tinh tay tu bon o:
  Precision = 17 / (17 + 3) = 0.8500
  Recall    = 17 / (17 + 13) = 0.5667
  F1        = 2 x 0.8500 x 0.5667 / (0.8500 + 0.5667) = 0.6800

Tinh bang scikit-learn:
  Precision = 0.8500
  Recall    = 0.5667
  F1        = 0.6800

Bao cao day du:
              precision    recall  f1-score   support

    khong bo     0.9536    0.9889    0.9709       270
      bo hoc     0.8500    0.5667    0.6800        30

    accuracy                         0.9467       300
   macro avg     0.9018    0.7778    0.8255       300
weighted avg     0.9432    0.9467    0.9418       300
```

4.1. Đọc kết quả

Tính tay và tính bằng thư viện ra cùng ba con số: precision 0.8500, recall 0.5667, F1 0.6800. Nghĩa là cứ 20 lời báo động thì 17 lời đúng, nhưng mô hình chỉ tìm ra 17 trên 30 bạn bỏ học.

Bảng `classification_report` in một dòng cho mỗi lớp:

- Dòng *khong bo* có recall 0.9889, vì gần như bạn học tiếp nào cũng được đoán đúng.
- Dòng *bo hoc* mới là dòng cần đọc trước, recall chỉ 0.5667.
- Cột `support` là số mẫu thật của mỗi lớp, 270 và 30.
- Dòng `macro avg` lấy trung bình cộng hai lớp, mỗi lớp nặng như nhau.
- Dòng `weighted avg` lấy trung bình theo số mẫu, nên bị lớp đông chi phối.

Khi nào ưu tiên precision, khi nào ưu tiên recall

Hỏi xem báo nhầm và bỏ sót, cái nào đắt hơn. Ở bài này, bỏ sót một bạn sắp bỏ học đắt hơn nhiều so với một buổi tư vấn thừa, nên ưu tiên recall. Với bộ lọc thư rác thì ngược lại: chặn nhầm một thư quan trọng rất phiền, nên ưu tiên precision.

5. Đổi ngưỡng quyết định

Hàm `predict` của hồi quy logistic báo bỏ học khi xác suất từ 0.5 trở lên. Nhưng 0.5 không phải luật bắt buộc. Lấy xác suất bằng `predict_proba` rồi tự đặt ngưỡng, ta điều khiển được mô hình báo nhiều hay báo ít.

```
# -*- coding: utf-8 -*-
"""Bước 4: đổi ngưỡng, và chọn ngưỡng đúng cách trên tập học."""

import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, precision_score, recall_score
from sklearn.model_selection import cross_val_predict, train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/sinh_vien_bo_hoc.csv")
X = df.drop(columns="bo_hoc")
y = df["bo_hoc"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

lr = make_pipeline(StandardScaler(), LogisticRegression())
lr.fit(X_tr, y_tr)
xs = lr.predict_proba(X_te)[:, 1]      # xác suất bỏ học của từng bạn

print("Tap kiem tra, doi nguong")
print("nguong  so bao   TP  FP  FN   precision  recall    F1")
for t in [0.5, 0.4, 0.3, 0.2, 0.1]:
    du = (xs >= t).astype(int)
    tp = int(((du == 1) & (y_te == 1)).sum())
    fp = int(((du == 1) & (y_te == 0)).sum())
    fn = int(((du == 0) & (y_te == 1)).sum())
    print(f"  {t:.1f}    {du.sum():4d}   {tp:3d} {fp:3d} {fn:3d}"
          f"    {precision_score(y_te, du):.4f}   "
          f"{recall_score(y_te, du):.4f}  {f1_score(y_te, du):.4f}")
print()

# Chọn ngưỡng đúng cách: chỉ dùng tập học, qua kiểm định chéo.
# Mỗi bạn trong tập học nhận xác suất từ một mô hình chưa từng thấy bạn ấy.
xs_hoc = cross_val_predict(lr, X_tr, y_tr, cv=5,
```

```
                              method="predict_proba")[:, 1]
cac_nguong = np.round(np.arange(0.05, 0.96, 0.05), 2)
f1_hoc = [f1_score(y_tr, (xs_hoc >= t).astype(int)) for t in cac_nguong]
t_tot = cac_nguong[int(np.argmax(f1_hoc))]
print(f"Nguong cho F1 cao nhat tren tap hoc: {t_tot}"
      f" (F1 kiem dinh cheo {max(f1_hoc):.4f})")

# Chọn xong mới dùng tập kiểm tra, và chỉ một lần
du = (xs >= t_tot).astype(int)
print(f"Tap kiem tra voi nguong {t_tot}:"
      f" precision {precision_score(y_te, du):.4f},"
      f" recall {recall_score(y_te, du):.4f},"
      f" F1 {f1_score(y_te, du):.4f}")
```

```
python code/h4_nguong.py
```

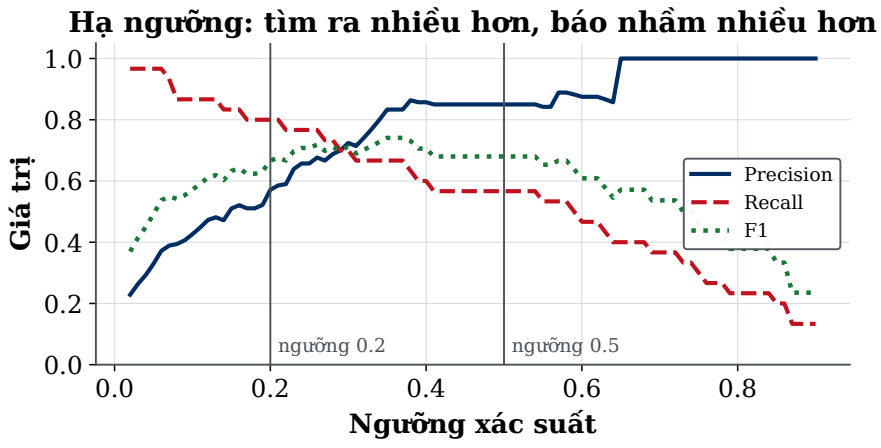
```
Tap kiem tra, doi nguong
nguong  so bao   TP  FP  FN   precision  recall    F1
  0.5       20    17   3  13    0.8500   0.5667  0.6800
  0.4       21    18   3  12    0.8571   0.6000  0.7059
  0.3       29    21   8   9    0.7241   0.7000  0.7119
  0.2       42    24  18   6    0.5714   0.8000  0.6667
  0.1       61    26  35   4    0.4262   0.8667  0.5714

Nguong cho F1 cao nhat tren tap hoc: 0.25 (F1 kiem dinh cheo 0.6473)
Tap kiem tra voi nguong 0.25: precision 0.6571, recall 0.7667, F1 0.7077
```

5.1. Đọc bảng thứ nhất

Hạ ngưỡng từ 0.5 xuống 0.1:

- Số bạn bị báo tăng từ 20 lên 61.
- Số bạn bỏ sót giảm từ 13 xuống 4, recall tăng từ 0.5667 lên 0.8667.
- Số báo động nhầm tăng từ 3 lên 35, precision giảm từ 0.8500 xuống 0.4262.



Hình 3: *Precision, recall và F1 trên tập kiểm tra theo ngưỡng xác suất.*

Không có ngưỡng nào làm precision và recall cùng cao nhất. Chọn ngưỡng là chọn một điểm đánh đổi, và điểm đó do bài toán thực tế quyết định.

5.2. Chọn ngưỡng đúng cách

Nhìn bảng thứ nhất, ngưỡng 0.3 cho F1 cao nhất trên tập kiểm tra, 0.7119. Nhưng nếu chọn ngưỡng bằng cách nhìn bảng đó, tập kiểm tra đã bị dùng để ra quyết định.

Phần thứ hai của mã làm đúng cách. Hàm `cross_val_predict` cho mỗi bạn trong tập học một xác suất đến từ mô hình chưa từng thấy bạn ấy, giống như kiểm định chéo. Trên các xác suất đó, ngưỡng 0.25 cho F1 cao nhất. Chọn xong mới mở tập kiểm tra ra một lần: precision 0.6571, recall 0.7667, F1 0.7077.

Hai con số F1 khác nhau một chút

F1 0.7077 thấp hơn 0.7119 một chút, nhưng đó là con số trung thực. Con số 0.7119 đẹp hơn chỉ vì ngưỡng được chọn sau khi đã nhìn đáp án.

6. Đường ROC và AUC

Thử mọi ngưỡng có thể, mỗi ngưỡng cho một điểm có hoành độ là tỷ lệ báo động nhầm và tung độ là recall. Nối các điểm lại được **đường ROC**.

Hai trục của đường ROC

$$\text{tỷ lệ báo động nhầm} = \frac{FP}{FP + TN} \qquad \text{recall} = \frac{TP}{TP + FN}$$

Đường càng ôm sát góc trên bên trái thì mô hình càng tốt. Diện tích dưới đường gọi là **AUC**. AUC bằng 1 là hoàn hảo, bằng 0.5 là không hơn đoán ngẫu nhiên.

```python
# -*- coding: utf-8 -*-
"""Bước 5: đường ROC, AUC, và so các mô hình đã học trong môn."""

import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, recall_score, roc_auc_score
from sklearn.metrics import roc_curve
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

df = pd.read_csv("data/sinh_vien_bo_hoc.csv")
X = df.drop(columns="bo_hoc")
y = df["bo_hoc"]
X_tr, X_te, y_tr, y_te = train_test_split(
```

```
    X, y, test_size=0.25, random_state=17, stratify=y
)

lr = make_pipeline(StandardScaler(), LogisticRegression())
xs = lr.fit(X_tr, y_tr).predict_proba(X_te)[:, 1]

# Mỗi ngưỡng cho một điểm của đường ROC
fpr, tpr, nguong = roc_curve(y_te, xs)
print("Duong ROC co", len(fpr), "diem. Vai diem dau:")
print("  bao dong nham   recall")
for a, b in list(zip(fpr, tpr))[:6]:
    print(f"      {a:.4f}      {b:.4f}")
print(f"AUC cua logistic = {roc_auc_score(y_te, xs):.4f}")
print()

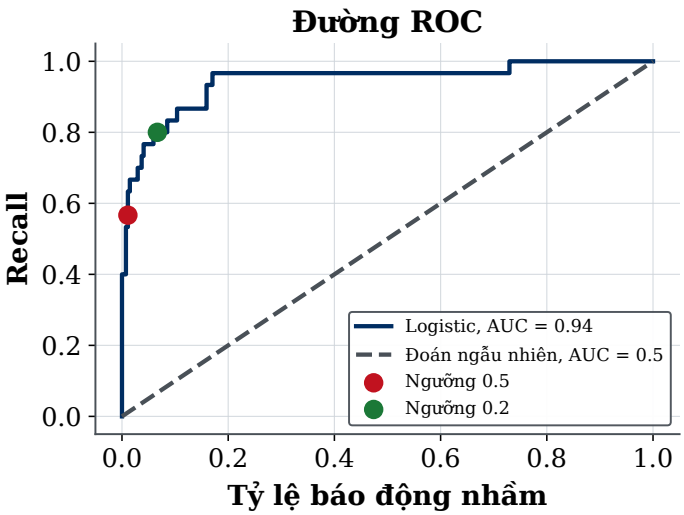
# So các mô hình đã học, cùng tập học, cùng tập kiểm tra
mo_hinh = {
    "Logistic": lr,
    "Naive Bayes": GaussianNB(),
    "KNN, k = 15": make_pipeline(StandardScaler(),
                                 KNeighborsClassifier(15)),
    "Cay sau 4": DecisionTreeClassifier(max_depth=4, random_state=0),
    "Rung 200 cay": RandomForestClassifier(200, random_state=0),
    "SVM RBF": make_pipeline(StandardScaler(),
                             SVC(probability=True, random_state=0)),
}
print("Mo hinh          AUC      recall    F1   (nguong 0.5)")
for ten, mh in mo_hinh.items():
    mh.fit(X_tr, y_tr)
    p = mh.predict_proba(X_te)[:, 1]
    du = (p >= 0.5).astype(int)
    print(f"  {ten:14s} {roc_auc_score(y_te, p):.4f}   "
          f"{recall_score(y_te, du):.4f}  {f1_score(y_te, du):.4f}")
```

```
python code/h5_roc_auc.py
```

```
Duong ROC co 28 diem. Vai diem dau:
  bao dong nham   recall
      0.0000      0.0000
      0.0000      0.0333
      0.0000      0.4000
      0.0074      0.4000
      0.0074      0.5333
      0.0111      0.5333
AUC cua logistic = 0.9449

Mo hinh          AUC      recall    F1   (nguong 0.5)
  Logistic       0.9449   0.5667  0.6800
  Naive Bayes    0.9336   0.3667  0.5116
```

```
KNN, k = 15     0.9274   0.2333  0.3684
Cay sau 4       0.8803   0.4667  0.5957
Rung 200 cay    0.9249   0.5667  0.6538
SVM RBF         0.8996   0.5000  0.6250
```



Hình 4: *Đường ROC của hồi quy logistic. Hai điểm đánh dấu là ngưỡng 0.5 và ngưỡng 0.2.*

6.1. Đọc kết quả

Đường ROC của logistic có 28 điểm. Ngay ở đầu đường, recall đã lên 0.4000 mà tỷ lệ báo động nhầm vẫn bằng 0, tức những bạn có xác suất cao nhất đều bỏ học thật. AUC của logistic là 0.9449.

Hiểu AUC bằng lời

Bốc ngẫu nhiên một bạn bỏ học và một bạn không bỏ học. AUC là xác suất mô hình cho bạn bỏ học điểm nguy cơ cao hơn. AUC không phụ thuộc ngưỡng, nên hợp để so các mô hình trước khi chọn ngưỡng.

6.2. So các mô hình đã học

Trên bộ dữ liệu này, hồi quy logistic có AUC cao nhất, 0.9449, và cả F1 cao nhất ở ngưỡng 0.5. Rừng ngẫu nhiên có cùng recall 0.5667 nhưng F1 thấp hơn. KNN có AUC 0.9274, khá tốt, nhưng ở ngưỡng 0.5 recall chỉ 0.2333.

AUC và recall kể hai câu chuyện khác nhau

KNN xếp hạng các bạn khá đúng, AUC cao, nhưng xác suất nó cho ra hiếm khi vượt 0.5 với lớp hiếm. Ở ngưỡng mặc định, nó báo quá ít. Với những mô hình như vậy, chọn lại ngưỡng như mục 5 là việc bắt buộc.

Mô hình đơn giản vẫn có thể thắng

Không có mô hình nào tốt nhất cho mọi bài toán. Ở đây các cột quan hệ khá thẳng với nguy cơ bỏ học, nên hồi quy logistic, mô hình đơn giản nhất, lại hợp nhất.

7. Kiểm định chéo nhiều thước đo và cân bằng lớp

Mục cuối gom ba việc thường làm khi đánh giá mô hình trên dữ liệu lệch lớp.

```python
# -*- coding: utf-8 -*-
"""Bước 6: kiểm định chéo với nhiều thước đo, và cân bằng lớp khi học."""

import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, precision_score, recall_score
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/sinh_vien_bo_hoc.csv")
X = df.drop(columns="bo_hoc")
y = df["bo_hoc"]

# Chia không phân tầng: số bạn bỏ học trong tập kiểm tra dao động
print("Khong stratify, so ban bo hoc trong tap kiem tra:")
dem = []
for rs in range(10):
    _, _, _, y_te_thu = train_test_split(X, y, test_size=0.25,
                                         random_state=rs)
    dem.append(int(y_te_thu.sum()))
print(" ", dem)
print()

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

lr = make_pipeline(StandardScaler(), LogisticRegression())
print("Kiem dinh cheo 5 phan tren tap hoc:")
for tm in ["accuracy", "precision", "recall", "f1"]:
    d = cross_val_score(lr, X_tr, y_tr, cv=5, scoring=tm)
    print(f"  {tm:10s} {d.mean():.4f}")
print()

# Cân bằng lớp: phạt nặng hơn khi đoán sai lớp hiếm
cb = make_pipeline(StandardScaler(),
                   LogisticRegression(class_weight="balanced"))
print("Tap kiem tra   precision  recall    F1")
for ten, mh in [("Binh thuong", lr), ("Can bang lop", cb)]:
```

```
    du = mh.fit(X_tr, y_tr).predict(X_te)
    print(f"  {ten:12s}  {precision_score(y_te, du):.4f}"
          f"    {recall_score(y_te, du):.4f}  {f1_score(y_te, du):.4f}")
```

```
python code/h6_can_bang.py
```

```
Khong stratify, so ban bo hoc trong tap kiem tra:
  [22, 26, 35, 29, 32, 27, 28, 20, 27, 23]

Kiem dinh cheo 5 phan tren tap hoc:
  accuracy   0.9289
  precision  0.7044
  recall     0.4810
  f1         0.5652

Tap kiem tra   precision  recall    F1
  Binh thuong   0.8500    0.5667  0.6800
  Can bang lop  0.4483    0.8667  0.5909
```

7.1. Vì sao phải chia có phân tầng

Không có `stratify`, mười cách chia ngẫu nhiên cho tập kiểm tra có từ 20 tới 35 bạn bỏ học. Recall tính trên 20 bạn và trên 35 bạn dao động rất khác nhau, nên kết quả khó so sánh. Với `stratify=y`, tập kiểm tra luôn có đúng 30 bạn bỏ học.

7.2. Kiểm định chéo với thước đo khác accuracy

Tham số `scoring` của `cross_val_score` nhận tên thước đo. Trên tập học, accuracy trung bình 0.9289 nghe rất cao. Nhưng recall trung bình chỉ 0.4810, tức mô hình để sót hơn một nửa số bạn bỏ học. Tham số `scoring` dùng được y hệt trong `GridSearchCV` để dò tham số theo recall hay F1.

7.3. Cân bằng lớp khi học

Tham số `class_weight="balanced"` làm mỗi lần đoán sai một bạn bỏ học bị phạt nặng hơn khi học. Kết quả trên tập kiểm tra: recall tăng từ 0.5667 lên 0.8667, precision giảm từ 0.8500 xuống 0.4483.

Giống hạ ngưỡng

Cân bằng lớp và hạ ngưỡng cùng đẩy mô hình về phía báo nhiều hơn, ít sót hơn. Recall 0.8667 ở đây đúng bằng recall của ngưỡng 0.1 ở mục 5. Chọn cách nào cũng được, miễn là quyết định dựa trên tập học chứ không dựa trên tập kiểm tra.

8. Bài tập

Sinh viên tạo thư mục `baitap07` nằm ngay trong thư mục bài thực hành. Vẫn chạy lệnh từ thư mục gốc, tức gõ `python baitap07/bai1.py`, và trong mã vẫn viết `data/...`. Bài nộp gồm tệp mã nguồn và ảnh chụp màn hình phần kết quả.

Bài tập 1: Tự viết hàm đánh giá

Viết hàm `danh_gia(y_that, y_doan)` trả về accuracy, precision, recall và F1, chỉ dùng numpy. Chạy hàm cho mô hình logistic và cho mô hình lười ở mục 2, so với các hàm của scikit-learn.
Với mô hình lười, precision có tính được không? Hàm của em xử lý trường hợp chia cho 0 thế nào?

Bài tập 2: Chữ số viết tay

Lấy lại SVM nhận dạng chữ số của bài 6, in `classification_report` cho cả 10 chữ số. Chữ số nào có recall thấp nhất? Chữ số nào có precision thấp nhất? Vì sao với bộ dữ liệu này, `macro avg` và `weighted avg` gần như bằng nhau?

Bài tập 3: Chọn ngưỡng theo yêu cầu

Nhà trường yêu cầu tìm ra ít nhất 80% số bạn bỏ học. Dùng xác suất từ `cross_val_predict` trên tập học như mục 5, tìm ngưỡng **lớn nhất** cho recall từ 0.8 trở lên.
Áp ngưỡng đó cho tập kiểm tra một lần. Recall có đạt 0.8 không? Precision và số bạn bị báo bằng bao nhiêu?

Bài tập 4: Tự vẽ đường ROC

Dùng `roc_curve` lấy hai mảng `fpr` và `tpr` của hồi quy logistic và của rừng ngẫu nhiên. Vẽ cả hai đường cùng đường chéo lên một hình bằng `plt.plot`, ghi AUC vào chú thích, lưu thành `roc.png`. Đường nào ôm sát góc trên bên trái hơn? Có đoạn nào hai đường cắt nhau không?

Bài tập 5: Cân bằng lớp cho mô hình khác

Thêm `class_weight="balanced"` cho `RandomForestClassifier` và cho `SVC`. So precision, recall, F1 trên tập kiểm tra trước và sau khi cân bằng. Mô hình nào thay đổi nhiều nhất?

Bài tập 6: MAE và RMSE với một lần đoán hỏng

Trên dữ liệu giá nhà của bài 1, tính MAE, RMSE và R^2 của hồi quy tuyến tính trên tập kiểm tra. Sau đó cộng thêm 5 tỷ vào dự đoán của đúng một căn, rồi tính lại ba con số.
Con số nào thay đổi nhiều nhất? Giải thích vì sao RMSE phạt nặng sai số lớn hơn MAE.

Tiêu chí chấm

Mã chạy được không lỗi, kết quả in ra đúng, có nhận xét bằng lời ở những bài yêu cầu, tên biến đặt có nghĩa. Bài 1 và bài 3 chấm nặng phần giải thích, vì đó là chỗ kiểm tra sinh viên có hiểu từng ô của ma trận nhầm lẫn và cách chọn ngưỡng không nhìn tập kiểm tra hay không.

9. Phụ lục A, bảng tra nhanh

Bảng 4: *Các hàm mới xuất hiện trong bài 7.*

Lệnh	Thư viện	Công dụng
`DummyClassifier`	scikit-learn	Mô hình lười, dùng làm mức sàn để so.
`confusion_matrix`	scikit-learn	Ma trận nhầm lẫn, hàng là nhãn thật, cột là nhãn đoán.
`precision_score`	scikit-learn	Precision của lớp dương.
`recall_score`	scikit-learn	Recall của lớp dương.
`f1_score`	scikit-learn	F1 của lớp dương, hoặc trung bình nhiều lớp với `average`.
`classification_report`	scikit-learn	Bảng precision, recall, F1 cho từng lớp.
`predict_proba`	scikit-learn	Xác suất của từng lớp, cột thứ hai là lớp 1.
`cross_val_predict`	scikit-learn	Dự đoán cho tập học bằng kiểm định chéo.
`roc_curve`	scikit-learn	Các điểm của đường ROC.
`roc_auc_score`	scikit-learn	Diện tích dưới đường ROC.
`scoring`	scikit-learn	Tên thước đo cho kiểm định chéo và dò lưới.
`class_weight`	scikit-learn	Đặt `"balanced"` để phạt nặng lỗi của lớp hiếm.

Bảng 5: *Các con số kết quả chính của bài thực hành.*

Đại lượng	Giá trị
Tỷ lệ bỏ học	0.099
Accuracy của mô hình lười và của logistic	0.9000 và 0.9467
TP, FP, FN, TN của logistic	17, 3, 13, 267
Precision, recall, F1 ở ngưỡng 0.5	0.8500, 0.5667, 0.6800
Ngưỡng chọn bằng kiểm định chéo	0.25
Precision, recall, F1 ở ngưỡng 0.25	0.6571, 0.7667, 0.7077
AUC của logistic	0.9449
Recall kiểm định chéo trên tập học	0.4810
Precision và recall khi cân bằng lớp	0.4483 và 0.8667

10. Phụ lục B, lỗi thường gặp

Bảng 6: *Các lỗi hay gặp riêng của bài đánh giá mô hình.*

Thông báo lỗi hoặc hiện tượng	Nguyên nhân và cách xử lý
`UndefinedMetricWarning: Precision is ill-defined`	Mô hình không báo mẫu nào là lớp dương, nên TP + FP bằng 0. Thêm `zero_division=0` nếu muốn coi precision bằng 0.
Precision và recall bị đổi chỗ cho nhau	Truyền ngược thứ tự tham số. Luôn viết nhãn thật trước, nhãn đoán sau: `recall_score(y_te, du_doan)`.
Số liệu các ô bị lệch	Trải ma trận sai thứ tự. Thứ tự đúng là `tn, fp, fn, tp = m.ravel()`.
Recall của lớp hiếm bằng 0 dù accuracy cao	Mô hình đang đoán mọi mẫu là lớp đông. Hạ ngưỡng hoặc dùng `class_weight="balanced"`.
`AttributeError: predict_proba is not available`	SVC mặc định không tính xác suất. Thêm `probability=True`, hoặc dùng `decision_function` cho `roc_auc_score`.
Tập kiểm tra có quá ít mẫu lớp hiếm	Quên `stratify=y` khi chia dữ liệu.
Điểm báo cáo đẹp hơn khi đem dùng thật	Đã chọn ngưỡng hoặc tham số bằng cách nhìn tập kiểm tra. Chọn lại bằng kiểm định chéo trên tập học.

Một lời khuyên chung

Trước khi báo cáo một con số, hãy trả lời ba câu hỏi. Mô hình lười đạt bao nhiêu? Loại sai nào đắt hơn? Tập kiểm tra đã bị dùng để ra quyết định nào chưa?